

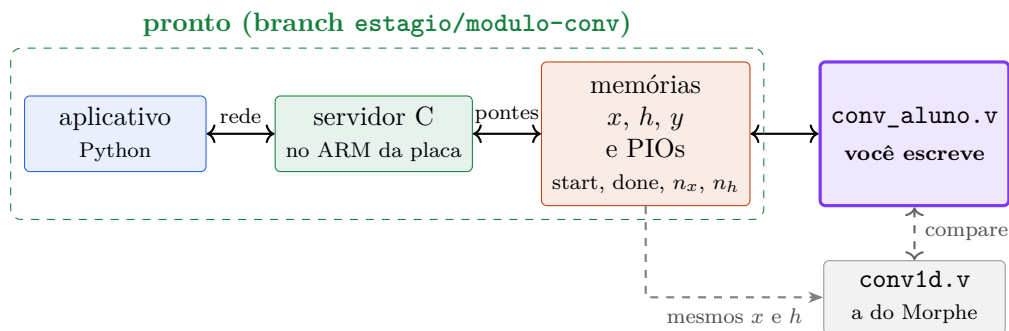
A convolução discreta na FPGA do Morphe

Roteiro prático — você escreve o módulo; o resto já está pronto

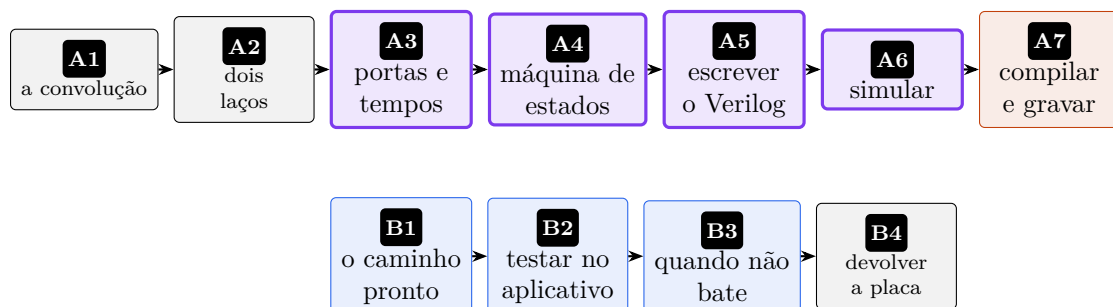
Antonio Nicassio Santos Lima Estágio LabPS/UEFS — supervisão: Prof. Armando S. Sanca

outubro de 2026

Neste roteiro você escreve, em Verilog, um bloco de hardware que calcula a **convolução discreta** de dois sinais e o compara com a convolução que o Morphe já tem na FPGA (o `conv1d.v`). Tudo em volta do seu módulo já está pronto — memórias, PIOs, servidor da placa e aplicativo. Você só escreve o corpo do `conv_aluno.v`:



- **Parte A** — desenvolver o módulo: a conta, a conta como tabela, a tabela como dois laços, os dois laços como máquina de estados, e a máquina em Verilog; depois simular e compilar.
- **Parte B** — testar no aplicativo: a janela **Convolução do aluno** manda os mesmos x e h para o seu módulo e para o `conv1d` e mostra a diferença amostra a amostra.



Antes de começar

Faça

Na estação ou num computador do laboratório, num terminal (digite `bash` primeiro: em `ksh` os comandos do Quartus não são encontrados). Um clone **seu**, na sua pasta, um comando por vez:

```
cd ~
```

```
git clone -b estagio/modulo-conv https://github.com/nicassiosantos/morphe.git minha-conv
```

```
cd minha-conv

git switch -c minha-conv
```

O que é e por quê

O `/opt/morphe` é a cópia que a turma usa: mexer nela muda o Morphe de todos. O clone na sua pasta é só seu, e a branch nova guarda o seu trabalho separado. A branch `estagio/modulo-conv` já tem tudo o que este roteiro chama de “pronto”.

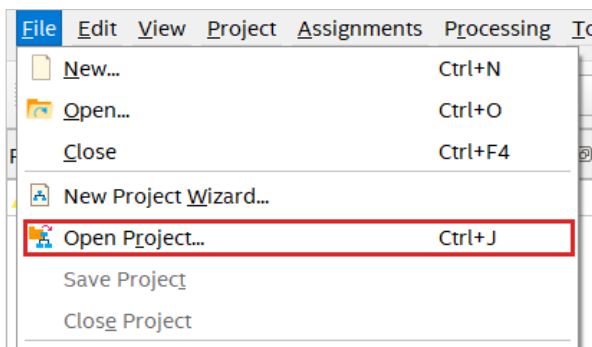
Cuidado

Copiar código do PDF pode juntar as linhas. Os mesmos trechos, em texto puro e na mesma ordem, estão em `docs/roteiro-conv/trechos.txt`.

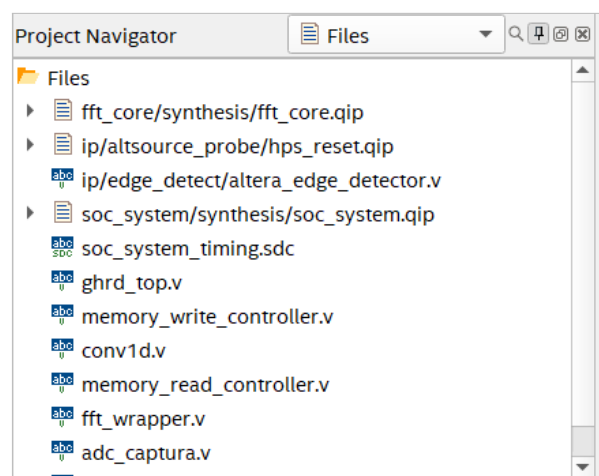
```
minha-conv/
├── Quartus/ ..... FPGA — abrir no Quartus 20.1
│   ├── conv_aluno.v ..... VOCÊ ESCRIVE — só as portas; o corpo é seu (A4)
│   ├── testbench/tb_conv_aluno.v ..... PRONTO — a simulação da A5
│   ├── conv1d.v ..... PRONTO — a convolução do Morphe, para comparar
│   ├── soc_system.qsys ..... PRONTO — memórias e PIOs do conv_aluno
│   ├── ghrd_top.v ..... PRONTO — o conv_aluno já ligado a eles
│   └── soc_system/, output_files/ ..... GERADO (não editar) na A6
├── C/ ..... servidor no ARM da placa
│   ├── morphe_server.c ..... PRONTO — opcode CONV_ALUNO
│   └── hps_0.h ..... GERADO (não editar) na A6 — os endereços
├── Python/ ..... aplicativo
│   └── conv_aluno_window.py ..... PRONTO — a janela da Parte B
└── docs/roteiro-conv/ ..... este roteiro e o trechos.txt
```

Faça

Abra o projeto no **Quartus 20.1** (menu de aplicativos, **Quartus Prime Lite 20.1**): **File** → **Open Project...** → `~/minha-conv/Quartus/soc_system.qpf`. No *Project Navigator*, troque **Hierarchy** por **Files** e dê duplo clique em `conv_aluno.v`.



File → Open Project...



Files: os arquivos do projeto

Cuidado

Sempre o 20.1. Se aparecer *IP upgrade recommended* ou *Used ... 23.1 (instead of 20.1)*, você abriu outra versão: feche **sem salvar**.

Parte A — desenvolver o módulo

A1 A convolução discreta

De onde vem a fórmula

O **impulso unitário** vale 1 num único instante:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \quad \delta[n - k] = \begin{cases} 1, & n = k \\ 0, & n \neq k \end{cases}$$

Qualquer sinal discreto é uma soma de impulsos deslocados, cada um pesado pelo valor do sinal naquele instante:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k] \delta[n - k]$$

Num sistema **linear e invariante ao deslocamento** (LID), chame de $h[n]$ a saída quando a entrada é $\delta[n]$ — a **resposta ao impulso**. Pela invariância, $\delta[n - k]$ produz $h[n - k]$; pela linearidade, a soma pesada de impulsos produz a mesma soma pesada de respostas. A saída para qualquer entrada é, então,

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{+\infty} x[k] h[n - k]$$

É isso que o seu módulo vai calcular: para cada n , uma soma de produtos.

A conta como tabela

Escreva $x[k]$ numa linha. Embaixo, uma linha para cada n com $h[n - k]$: é o h **invertido** ($h[-k]$) e **deslizado** n casas para a direita. Multiplique coluna a coluna e some a linha: dá $y[n]$. Por exemplo, com $x = \{1, 2, 3, 1\}$ e $h = \{1, 2, 1, -1\}$, os dois começando em $n = 0$:

$x[k]$	$k = 0$	$k = 1$	$k = 2$	$k = 3$		$k_{\min}$	$k_{\max}$
	1	2	3	1			
$n = 0: h[0 - k]$	1	0	0	0	$y[0] = 1 \cdot 1 = \mathbf{1}$	0	0
$n = 1: h[1 - k]$	2	1	0	0	$y[1] = 2 + 2 = \mathbf{4}$	0	1
$n = 2: h[2 - k]$	1	2	1	0	$y[2] = 1 + 4 + 3 = \mathbf{8}$	0	2
$n = 3: h[3 - k]$	-1	1	2	1	$y[3] = -1 + 2 + 6 + 1 = \mathbf{8}$	0	3
$n = 4: h[4 - k]$	0	-1	1	2	$y[4] = -2 + 3 + 2 = \mathbf{3}$	1	3
$n = 5: h[5 - k]$	0	0	-1	1	$y[5] = -3 + 1 = \mathbf{-2}$	2	3
$n = 6: h[6 - k]$	0	0	0	-1	$y[6] = \mathbf{-1}$	3	3

A mesma tabela, escrita como produto de matriz por vetor:

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \\ y[3] \\ y[4] \\ y[5] \\ y[6] \end{bmatrix} = \begin{bmatrix} h[0] & 0 & 0 & 0 \\ h[1] & h[0] & 0 & 0 \\ h[2] & h[1] & h[0] & 0 \\ h[3] & h[2] & h[1] & h[0] \\ 0 & h[3] & h[2] & h[1] \\ 0 & 0 & h[3] & h[2] \\ 0 & 0 & 0 & h[3] \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix}, \quad H_{n,k} = h[n-k]$$

Cada diagonal da matriz repete o mesmo valor (é uma matriz de **Toeplitz**), e cada $y[n]$ é o produto de uma linha dela por x .

O que é e por quê

Três fatos da tabela que mandam no projeto do hardware:

- **tamanho da saída:** com n_x amostras de x e n_h de h , há $n_x + n_h - 1$ linhas; y vai de $n = 0$ a $n = n_x + n_h - 2$ (no exemplo, $4 + 4 - 1 = 7$);
- **as casas em cinza não custam nada:** lá $x[k]$ ou $h[n-k]$ não existe e o produto é zero. Só as casas coloridas, de $k_{\min}$ a $k_{\max}$, precisam de conta;
- **uma linha de cada vez:** cada linha só precisa de um acumulador. Somou, gravou $y[n]$, zera e passa para a próxima.

Já está pronto

A origem do tempo. Na memória, x e h começam no endereço 0, mesmo que no papel comecem em outro n . Se x começa em $n = a$ e h em $n = b$, y começa em $n = a + b$: o aplicativo faz essa conta na hora de desenhar. O seu módulo não precisa saber disso.

A2 Da tabela a dois laços

As casas coloridas da linha n são as colunas onde $x[k]$ e $h[n-k]$ existem ao mesmo tempo:

$$\underbrace{0 \leq k \leq n_x - 1}_{x[k] \text{ existe}} \quad \text{e} \quad \underbrace{0 \leq n - k \leq n_h - 1}_{h[n-k] \text{ existe}} \iff n - n_h + 1 \leq k \leq n$$

$$k_{\min} = \max(0, n - n_h + 1) \quad k_{\max} = \min(n, n_x - 1)$$

Confira nas duas últimas colunas da tabela. Com isso, a convolução inteira é:

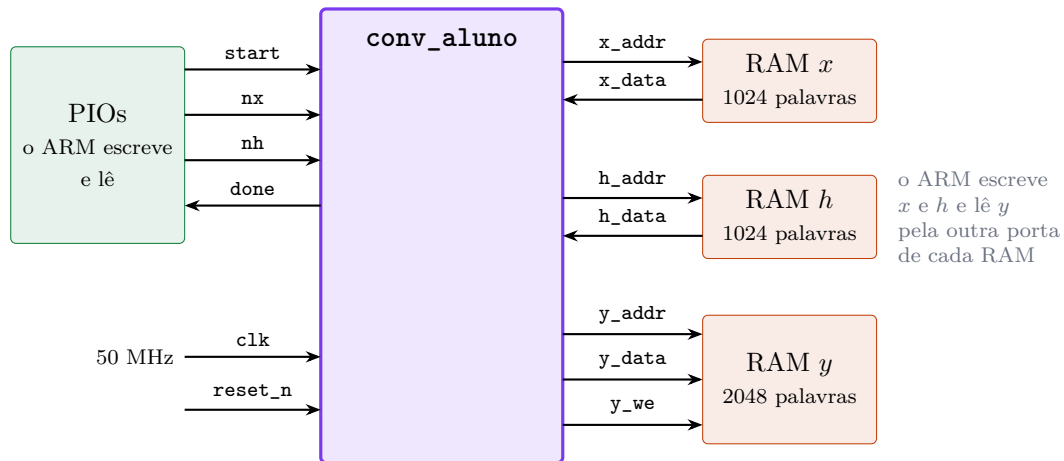
```
para n = 0 até nx + nh - 2:           # uma LINHA da tabela
    acc = 0
    para k = kmin(n) até kmax(n):     # as casas coloridas
        acc = acc + x[k] * h[n - k]
    y[n] = acc                       # a soma da linha
```

Pense

Faça a tabela com $x = \{2, 1\}$ e $h = \{1, 1, 1\}$. Quantas linhas? Quais $k_{\min}$ e $k_{\max}$ em cada uma? Em alguma linha $k_{\min} > k_{\max}$? (Resposta: 4 linhas, $y = \{2, 3, 3, 1\}$; nunca, porque toda linha tem ao menos uma casa colorida.) E quantos produtos ao todo para $n_x = 1024$ e $n_h = 32$?

A3 O que entra e o que sai do módulo

O `conv_aluno.v` já tem as portas. Não mude nomes nem larguras: é por eles que o topo (`ghrd_top.v`) o liga às memórias e aos PIOs.



```

module conv_aluno #(
    parameter DATA_WIDTH = 32,
    parameter X_ADDR_BITS = 10,      // x: ate 1024 amostras
    parameter H_ADDR_BITS = 10,      // h: ate 1024 amostras
    parameter Y_ADDR_BITS = 11       // y: ate 2047 amostras (nx + nh - 1)
)(
    input wire clk,                  // 50 MHz
    input wire reset_n,              // ativo em 0

    // --- controle (PIOs) ---
    input wire start,                // HPS -> FPGA
    output reg done,                 // FPGA -> HPS
    input wire [X_ADDR_BITS-1:0] nx, // amostras de x, 1 a 1024
    input wire [H_ADDR_BITS-1:0] nh, // amostras de h, 1 a 1024

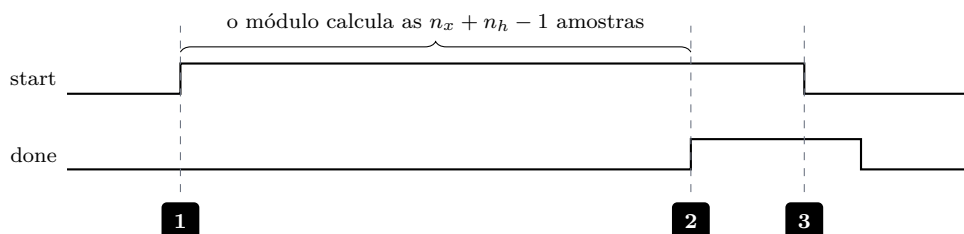
    // --- memoria de x[k] (so leitura) ---
    output reg [X_ADDR_BITS-1:0] x_addr,
    input wire signed [DATA_WIDTH-1:0] x_data,

    // --- memoria de h[k] (so leitura) ---
    output reg [H_ADDR_BITS-1:0] h_addr,
    input wire signed [DATA_WIDTH-1:0] h_data,

    // --- memoria de y[n] (so escrita) ---
    output reg [Y_ADDR_BITS-1:0] y_addr,
    output reg signed [DATA_WIDTH-1:0] y_data,
    output reg y_we
);

```

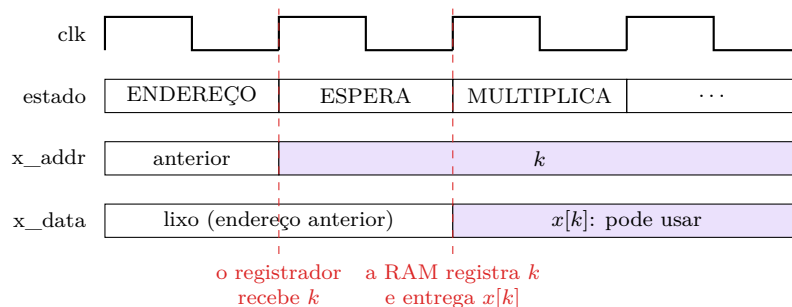
O start e o done



- 1** o ARM escreve x , h , n_x e n_h e sobe o **start**: é a **borda de subida** que dispara (guarde o **start** do ciclo anterior num registrador e compare);
- 2** terminou de gravar y : **done** = 1, e fica em 1;
- 3** o ARM leu y e baixou o **start**: **done** volta a 0 e o módulo espera o próximo **start**.

Ler a memória leva um ciclo

A memória é **síncrona**: ela registra o endereço na borda do relógio e só entrega o dado depois dela. Se o `x_addr` é um registrador que você muda num estado, o dado certo só pode ser usado **dois estados depois** — por isso a máquina da A4 tem um estado de ESPERA no meio:

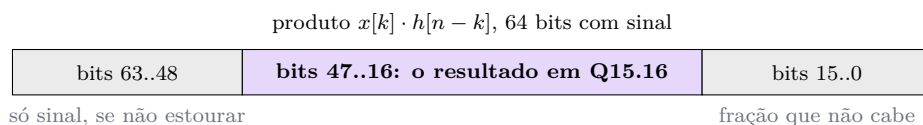


A escrita de y é mais simples: no ciclo em que `y_we = 1`, a memória grava `y_data` em `y_addr`. Ponha os três no mesmo estado e baixe o `y_we` no seguinte.

Multiplicar em Q15.16

O que é e por quê

A FPGA não tem ponto flutuante. O Morphe guarda cada número real como um inteiro de 32 bits, o valor vezes 2^{16} (**Q15.16**): $1,0 = 0x00010000$, $-1,0 = 0xFFFF0000$. Na soma, a vírgula não mexe. No **produto**, os dois fatores trazem 2^{16} cada, e o resultado sai com 2^{32} : 32 bits de fração. Para voltar a Q15.16, jogue fora os 16 bits de baixo e fique com os 32 seguintes.



Cuidado

Para bater bit a bit com o conv1d (a Parte B compara *igualdade*, não “parecido”), faça a conta como ele faz:

- cada produto em 64 bits, com sinal: `x_data` e `h_data` já são **signed**, e o registrador que recebe o produto também precisa ser (`reg signed [63:0]`);
- de **cada** produto, os bits `[47:16]`; some esses valores, não os produtos inteiros;
- acumulador de **32 bits**, sem saturação.

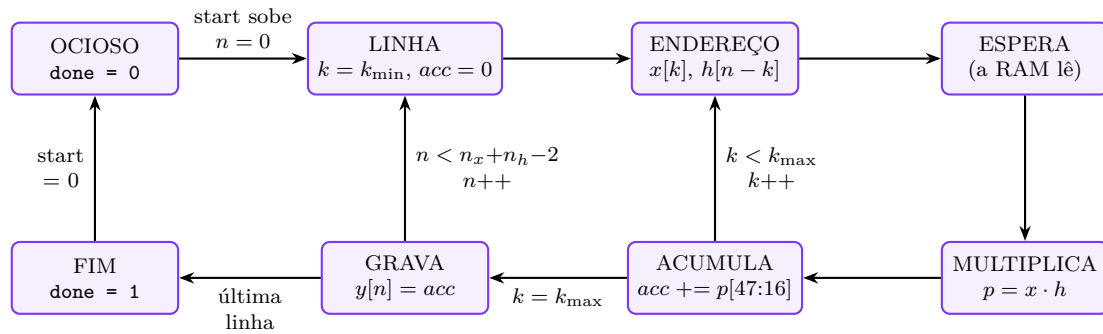
Somar os 64 bits e cortar só no fim é até mais preciso, mas dá diferenças de 1 ou 2 unidades na última casa em relação ao `conv1d`.

A4 Planejar a máquina de estados

O que é e por quê

Em software, os dois laços rodam um comando depois do outro. Em hardware, tudo acontece na **borda do relógio**, e cada coisa que depende de outra precisa de um ciclo próprio: ler a memória leva um ciclo (A3), multiplicar 32×32 bits e somar no mesmo ciclo deixa o circuito lento demais. A **máquina de estados** é o jeito de dizer “primeiro isto, no ciclo seguinte aquilo”: um registrador guarda em que passo o módulo está, e um **case** diz o que fazer em cada passo e qual é o próximo.

Para desenhar a máquina, percorra o pseudocódigo da A2 e pergunte, em cada linha: *isto cabe num ciclo? depende de algo que só fica pronto no ciclo anterior?* Um caminho possível (há outros):



Estado	O que faz (registradores que mudam)	Próximo
OCIOSO	done = 0; na borda do start, $n = 0$	LINHA, se o start subiu
LINHA	$k = k_{\min}(n)$, $k_{\max} = k_{\max}(n)$, $acc = 0$	ENDEREÇO
ENDEREÇO	$x_addr = k$, $h_addr = n - k$	ESPERA
ESPERA	nada: a memória está lendo	MULTIPLICA
MULTIPLICA	$p = x_data \times h_data$ (64 bits)	ACUMULA
ACUMULA	$acc = acc + p[47:16]$; se ainda há coluna, $k = k + 1$	ENDEREÇO ou GRAVA
GRAVA	$y_addr = n$, $y_data = acc$, $y_we = 1$; se ainda há linha, $n = n + 1$	LINHA ou FIM
FIM	done = 1	OCIOSO, quando o start cai

Na tabela da A1	Na máquina
começar uma linha	LINHA
uma casa colorida	ENDEREÇO → ESPERA → MULTIPLICA → ACUMULA: 4 ciclos
somar a linha	o próprio acumulador, casa a casa
escrever $y[n]$	GRAVA

Pense

Com 4 ciclos por produto e 50 MHz, quanto tempo leva a maior convolução (1024×1024 , cerca de um milhão de produtos)? Daria para gastar menos ciclos por produto? (Dica: enquanto a memória lê a coluna $k + 1$, o produto da coluna k poderia estar sendo feito.)

A5 Escrever o Verilog, por partes

Tudo vai no `conv_aluno.v`, entre o `)`; das portas e o `endmodule`. Escreva na ordem abaixo; a cada parte, salve (Ctrl+S).

1. Os registradores

Antes de escrever código, decida o que precisa ser lembrado de um ciclo para o outro, e de que tamanho:

Registrador	Bits	Sinal	Por quê
estado	3	não	8 estados cabem em 3 bits
n	13	sim	vai até 2046 (11 bits); com sinal porque entra em $n - n_h + 1$, que pode ser negativo
$k, k_{\max}$	13	sim	na mesma conta que n
acumulador	32	sim	Q15.16, como y
produto	64	sim	32×32 bits dá 64
start anterior	1	não	para achar a borda de subida

Cuidado

Se n , k e n_h forem **sem sinal**, $n - n_h + 1$ com $n < n_h - 1$ não dá negativo: dá a volta e vira um número enorme, e o $\max(0, \dots)$ escolhe o número enorme. Declare `reg signed [12:0]` e passe n_x e n_h para a mesma largura com sinal: `$signed({2'b00, nx})`.

2. Os limites como fios

$k_{\min}$ e $k_{\max}$ dependem só de n , n_x e n_h : não precisam de estado, são **fios** (wire) calculados o tempo todo. Máximo e mínimo se escrevem com o operador condicional: $(a > b) ? a : b$ vale o maior entre a e b . Escreva dois wire `signed [12:0]`, um para cada fórmula da A2, e use-os no estado LINHA.

3. O esqueleto

A borda do **start**, o reset e dois dos oito estados. O resto é com você, seguindo a tabela da A4:

```
// os estados
localparam [2:0] S_OCIOSO    = 3'd0;
localparam [2:0] S_LINHA     = 3'd1;
localparam [2:0] S_ENDERECO  = 3'd2;
localparam [2:0] S_ESPERA    = 3'd3;
localparam [2:0] S_MULTIPlica = 3'd4;
localparam [2:0] S_ACUMULA    = 3'd5;
localparam [2:0] S_GRAVA     = 3'd6;
localparam [2:0] S_FIM       = 3'd7;

reg [2:0] estado;

// borda de subida do start: 1 so no ciclo em que ele passa de 0 a 1
reg start_d;
wire start_pulso = start & ~start_d;
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) start_d <= 1'b0;
    else          start_d <= start;
end

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        estado <= S_OCIOSO;
        done   <= 1'b0;
        y_we   <= 1'b0;
        // zere aqui tambem os seus registradores
    end else begin
        y_we <= 1'b0;          // so fica em 1 no ciclo de GRAVA

        case (estado)
            S_OCIOSO: begin
                done <= 1'b0;
                if (start_pulso) begin
```

```

        n      <= 0;
        estado <= S_LINHA;
    end
end

// S_LINHA, S_ENDERECO, S_ESPERA, S_MULTIPLICA,
// S_ACUMULA e S_GRAVA: escreva aqui

S_FIM: begin
    done <= 1'b1;    // fica em 1 ate o HPS baixar o start
    if (!start) estado <= S_OCIOSO;
end

    default: estado <= S_OCIOSO;
endcase
end
end

```

O que é e por quê

- $\leq$ (atribuição *não bloqueante*): todos os registradores mudam juntos na borda, com os valores de antes dela. Num estado, $k \leq k + 1$ e $x_addr \leq k$ usam o k **antigo** — é assim que o hardware funciona, e é por isso que cada passo que depende do anterior vai num estado próprio.
- $y_we \leq 1'b0$ antes do `case` vale para todos os estados; o GRAVA o sobrescreve com 1. Assim y_we nunca fica esquecido em 1.

4. Os seis estados que faltam

Faça

Escreva um `S_...: begin ... end` para cada linha da tabela da A4. Pontos de atenção:

- **ENDERECO**: o endereço de h é $n - k$, calculado em 13 bits e só depois cortado para os 10 de h_addr (um wire signed [12:0] com $n - k$ e $h_addr \leq n_menos_k[9:0]$);
- **MULTIPLICA**: produto $\leq x_data * h_data$; — os dois já são signed, e o produto também precisa ser;
- **ACUMULA**: some produto[47:16], não o produto inteiro; compare k com $k_{\max}$ **antes** de incrementar;
- **GRAVA**: a última linha é $n = n_x + n_h - 2$.

Para depurar: um `$display` dentro de um estado imprime na simulação (A6) e é ignorado pelo Quartus. Por exemplo, no GRAVA:

```
$display("y[%0d] = %h", n, acc);
```

A6 Simular antes de compilar

O que é e por quê

Compilar o projeto leva de 10 a 15 minutos; simular leva um segundo. O `testbench/tb_conv_aluno.v` imita as três memórias (com o mesmo ciclo de leitura da A3), dispara o seu módulo como o servidor faz e compara cada $y[n]$ com a conta feita do jeito do `conv1d`. São sete casos: um caso pequeno de inteiros, uma amostra de cada lado, h maior que x , valores fracionários quaisquer, os dois extremos de 1024 amostras e uma segunda rodada seguida.

Faça

De dentro de ~/minha-conv/Quartus:

```
iverilog -g2005 -o /tmp/tb_conv.vvp testbench/tb_conv_aluno.v conv_aluno.v
```

```
vvp /tmp/tb_conv.vvp
```

Confira

Cada caso numa linha começando com ok, e a última linha é TUDO OK:

```
ok    caso pequeno 4 x 4  (nx=4 nh=4, 81 ciclos)
ok    1 x 1, fracao e sinal (nx=1 nh=1, 9 ciclos)
ok    h maior que x  (nx=3 nh=6, 91 ciclos)
ok    aleatorio 50 x 17  (nx=50 nh=17, 3535 ciclos)
ok    x com 1024 amostras (nx=1024 nh=3, 14343 ciclos)
ok    h com 1024 amostras (nx=3 nh=1024, 14343 ciclos)
ok    segunda rodada seguida (nx=4 nh=4, 81 ciclos)
TUDO OK
```

O número de ciclos do seu pode ser outro: depende de quantos estados você usou. O que tem de bater são os valores.

O testbench diz

Provável causa

done nao subiu	o módulo não sai de um estado, ou não detecta a borda do start
done nao voltou a 0	falta voltar ao OCIOSO quando o start cai
valores deslocados ($y[n]$ tem a conta de outra casa)	usou o dado um estado cedo demais: falta a ESPERA
erra só nas primeiras ou nas últimas linhas	$k_{\min}$ ou $k_{\max}$ errado; índice sem sinal com $n - n_h + 1 < 0$
diferença de 1 na última casa	cortou os bits no fim, e não em cada produto
escrito fora da saida	gravou mais de $n_x + n_h - 1$ amostras, ou y_we ficou em 1
caso pequeno: y nao e ...	a conta está errada até com inteiros: refaça a tabela da A1 à mão (é o mesmo x e h)

Cuidado

Se o computador não tiver o iverilog, peça ao responsável pelo laboratório. Sem simular, cada erro custa uma compilação inteira.

A7 Compilar e gravar a placa

Já está pronto

As memórias conv_aluno_x, _h, _y e os PIOs conv_aluno_start, _done, _nx, _nh já estão no soc_system.qsys, com endereço travado; o ghrd_top.v já os liga ao seu módulo. Falta só **gerar** o sistema uma vez e compilar.

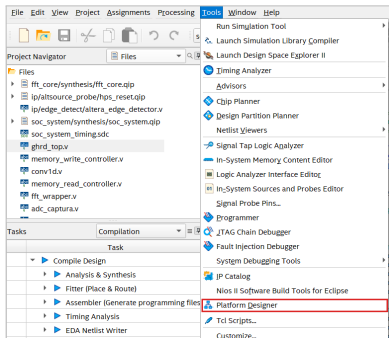
Faça

1. **Tools** → **Platform Designer** → soc_system.qsys → **Generate** → **Generate HDL...** → **Generate**. Espere *Generate Completed*, **Finish** e feche o Platform Designer **sem mudar nada**.
2. No Quartus: **Processing** → **Start Compilation** (Ctrl+L).
3. No terminal, de ~/minha-conv/Quartus:

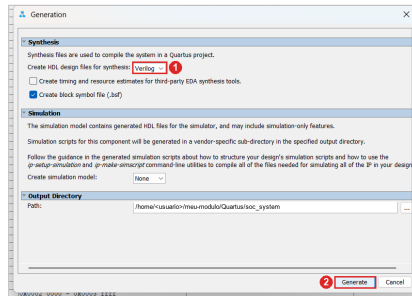
```
python3 gen_hps_header.py
```

4. Na **estação**, de ~/minha-conv, fora do horário de aula. No lugar de CABO, o nome do cabo da sua placa (o jtagconfig lista; se houver mais de um, desligue o USB-Blaster da sua placa, rode de novo e veja qual sumiu):

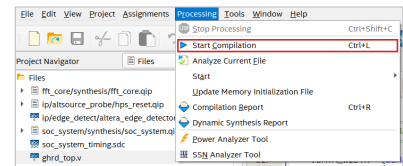
```
./morphe-up.sh --cable 'CABO' --deploy
```



Tools → Platform Designer



Generate HDL



Start Compilation

O que é e por quê

Gerar escreve o Verilog do sistema e o `.sopcinfo`. **Compilar** junta isso ao seu módulo e produz o `.sof`, a configuração da FPGA. O `gen_hps_header.py` tira os endereços do `.sopcinfo` para o `C/hps_0.h`, e o `--deploy` recompila o servidor da placa com ele: servidor e FPGA sempre andam juntos.

Confira

- *Full Compilation was successful*, e no **Compilation Report** → **Timing Analyzer**, *Slack* positivo no `clock_50_1`;
- `grep CONV_ALUNO ../C/hps_0.h` mostra `0x43000`, `0x44000`, `0x46000` e `0x180` a `0x1b0`;
- o `morphe-up.sh` termina com os testes 4/4.

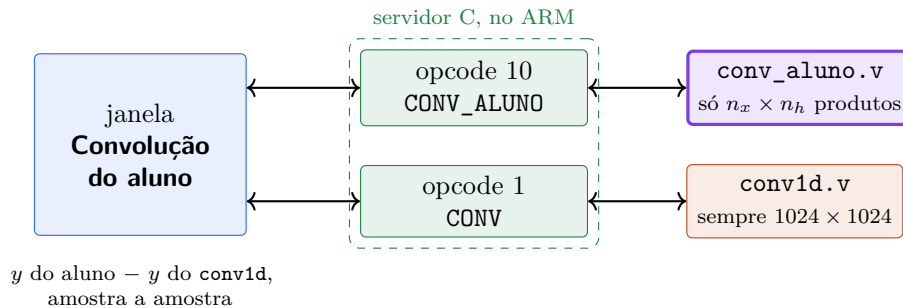
Cuidado

Erro de compilação que aponta para o `conv_aluno.v` é seu: o *Messages*, embaixo, diz a linha. O Quartus é mais rigoroso que o `iverilog` com larguras; leia os *Warnings* do seu arquivo também.

Parte B — testar no aplicativo

B1 O caminho que já está pronto

O aplicativo manda os **mesmos** x e h por dois caminhos — o seu módulo e o `conv1d` — e compara as respostas:



Peça (pronta)	O que faz
Python/conv_aluno_window.py	a janela: gera x e h , passa para Q15.16, faz os dois pedidos numa <i>thread</i> e desenha y e a diferença
Python/morphe_protocol.py	OP_CONV_ALUNO = 10: o pedido tem o mesmo formato da convolução (cabeçalho, x , depois h)
C/morphe_server.c: handle_conv_aluno	escreve x , h , n_x e n_h na FPGA, zera y , sobe o <code>start</code> , espera o <code>done</code> (até 5 s), lê as $n_x + n_h - 1$ amostras de y , baixa o <code>start</code> e responde
C/morphe_server.c: conferência do <i>sysid</i>	recusa o pedido se o <code>.sof</code> da placa não for o que tem o <code>conv_aluno</code> : a placa não trava
Quartus/ghrd_top.v	liga as portas do seu módulo às memórias (<code>chipselct</code> e <code>clken</code> sempre em 1) e aos PIOs

B2 Testar no aplicativo

Faça

Num computador do laboratório, de `~/minha-conv`:

```
python3 Python/morphe_app.py
```

Em *Placas do laboratório*, escolha a **sua** placa. No menu, **Convolução do aluno (conv_aluno × conv1d)**. Em cada painel, escolha o sinal e clique em **Gerar**; depois, **Comparar na FPGA**.

Vá do caso mais simples ao maior. Cada um testa uma parte do módulo:

$x[n]$	$h[n]$	o que esperar, e o que testa
impulso, $N = 1$	impulso, $N = 1$	$y = \{1\}$: uma linha, uma casa
impulso, $N = 8$	retangular ou degrau, $N = 5$	y repete o h : $\delta[n] * h[n] = h[n]$
degrau, $N = 8$	degrau, $N = 8$	um triângulo de 15 amostras: $k_{\min}$ e $k_{\max}$ nas linhas do começo e do fim
degrau, $N = 3$	exponencial, $N = 20$	h maior que x : os limites trocam de papel
senóide, $N = 1024$	degrau, $N = 32$	soma móvel de 32 amostras: y suaviza x ; endereços até o fim da memória
exponencial, $N = 1024$	senóide, $N = 1024$	o maior caso: 2047 amostras, cerca de um milhão de produtos

Confira

- no terceiro gráfico, as hastes (seu módulo) e os $\times$ (o `conv1d`) coincidem;
- o gráfico de baixo, a diferença, é zero em todas as amostras;
- a barra de baixo diz *iguais, bit a bit*.

B3 Quando não bate

Se as respostas diferem, a barra de baixo diz em que n está a **primeira** diferença e quantas amostras cada módulo acerta contra a mesma conta feita no computador — o que separa um erro seu de um erro do `conv1d`. O gráfico da diferença mostra *onde* está o erro:

O aplicativo mostra	Provável causa
<i>timeout</i>	o <code>done</code> não subiu em 5s: o módulo travou num estado, ou não viu a borda do <code>start</code>
y todo zero	o módulo não grava: <code>y_we</code> nunca sobe (o servidor zera y antes de cada pedido)
funciona uma vez, e na segunda dá <i>timeout</i>	o módulo não volta ao OCIOSO quando o <code>start</code> cai
diferença só no começo e no fim de y	$k_{\min}$ ou $k_{\max}$ errado; conta sem sinal com $n - n_h + 1 < 0$
y parecido, mas deslocado ou “borrado”	usou o dado da memória um ciclo cedo demais: falta a ESPERA
diferença de ± 1 ou ± 2 espalhada	cortou os bits só no fim, e não em cada produto
certo em sinais pequenos, errado em $N = 1024$	largura de algum registrador ou endereço pequena demais

Cuidado

Antes de mexer no módulo, reproduza o erro no testbench (A6) — se o testbench passa e a placa erra, compare os dois casos: tamanho, sinal dos valores, rodadas seguidas. Cada volta pela placa custa uma compilação.

B4 Devolver a placa à turma

Faça

Sempre, ao terminar. Na estação, de `~/minha-conv`:

```
./morphe-up.sh --down
```

e depois, de `/opt/morphe`, com o mesmo CABO da A7:

```
cd /opt/morphe
```

```
./morphe-up.sh --cable 'CABO'
```

Erros comuns

Sintoma	Causa e solução
<i>IP upgrade recommended</i> , ou <i>Used ... 23.1</i>	abriu com outro Quartus; feche sem salvar e abra o 20.1
<i>can't find port</i> ao compilar	mudou o nome ou a largura de uma porta do <code>conv_aluno.v</code> ; volte à da A3
<i>Slack</i> negativo no <code>clock_50_1</code>	conta demais num ciclo só (multiplicar <i>e</i> somar no mesmo estado, por exemplo); separe em dois estados
o aplicativo diz <i>opcode desconhecido</i>	o servidor da placa é o antigo: faltou o <code>--deploy</code> da A7, ou o app está ligado em outra placa
o aplicativo diz <i>bitstream ... nao e o deste servidor</i>	a placa está com outro <code>.sof</code> : grave o seu (A7)
nunca use System → Assign Base Addresses	muda o endereço de todos os componentes, e o servidor passa a escrever no lugar errado